AGNIRATH

STRATEGY MODULE

RECRUITMENT'26

APPLICATION FOR THE POSITION OF CREW MEMBER

AGNIRATH

GENERAL INSTRUCTIONS

- Try to answer as many questions as possible.
- You can use the help of any online resources to answer the questions.
- We have attached a few references (hyperlinks in questions) which will be helpful to solve the questions.
- Try to be creative but feasible with your ideas.
- Explain your ideas/strategy wherever possible.
- Be as concise and clear as possible.
- Write your answers in points instead of paragraphs. In case of any doubts, feel free to reach out to the respective heads.
- In case you need any sort of data, do not be afraid to message us.
- Save your answer script as a PDF file with the name <moduleacronym>_<roll number>_<name>.pdf,Eg, ST_AE24xxxx_SAKSHAM.pdf (ST for Strategy Module)
- Application submissions are to be made through Agnirath Website. Details of which will be shared on Whatsapp group later on.
- The deadline for submission is **19th April 2026**

STRATEGY

Saksham - +91 734 905 6529

Hafiz - +91 953 979 0357

AGNIRATH

CONTENT

This application consists of 5 sections:

1. Foundation

In this section, you will derive the physical equations that govern our race. From aerodynamic drag and motor efficiencies to solar irradiance geometry and battery degradation, this is where you prove you understand the raw physics of energy management.

2. Taming the Telemetry

Raw data is messy and deceptive. This section tests your ability to clean, optimize and extract signal from the noise using standard data libraries.

3. Staying in Control

Feedback. Control. Adaptability. From PID to Model Predictive Control, this section probes how well you can tame dynamic systems.

4. The Strategist's Crucible

A textbook mathematician optimizes for the average; a strategist optimizes for survival. This section pushes your intuition to the limit, forcing you to balance time, battery budgets and everything in between.

5. The Final Challenge

Implementation is prioritised over ideal design. Given a full-blown predictive optimization scenario on the Sasol route, this section tests your capabilities to bring dreams to reality.

We understand the application is long and will take time and effort. Keep the answers to the point and we value the creativity of your ideas.

Good luck and hope you have a fun time!

FOUNDATION

AGNIRATH

Question 1

The Invisible Enemies (Forces & Power)

When the car is moving on the highway, the universe is actively trying to slow it down. We need to quantify this resistance before we can fight it.

1. Identify the primary physical forces acting against the car when moving up a hill at a constant velocity. Write down the mathematical expressions for each.
2. In strategy, we care more about Power than Force. Convert your three force equations into Power equations.
3. The Intuition Check: Looking at your Power equations, explain why driving at 90 km/h requires vastly more than double the power of driving at 45 km/h. Who is responsible for this?

Question 2

Chasing the Sun

Your battery is just a buffer, we know that the Sun is your actual fuel tank. But the Sun moves and the photons don't always hit exactly where you want them to.

1. Solar radiation isn't just one simple beam. Define DNI, DHI and GHI. Find other useful terms that describe solar irradiation
2. If thick cloud cover suddenly rolls in, how does the ratio of DNI to DHI change and why does this matter for a curved solar canopy?
3. Arrive at a comprehensive mathematical formula to compute the actual solar power incident on a flat solar panel at a given time of day. Your formula must include the panel's area, efficiency and the geometric relationship between the Sun and the panel.

Question 3

The Leaky Bucket (Battery & Electrical Health)

Before your solar chariot turns into a very expensive paperweight, you'll need to understand the Battery as a Strategist. Let's break down the drama inside the cells:

1. How do you visualize the "percentage" of a battery?

2. Over thousands of kilometers, the chemistry degrades. What parameter is used to indicate the "health" of a battery? Write an expression defining the actual usable Capacity of the battery as a function of its original Rated Capacity and the health parameter.

Question 4

Motor & Regen Braking

Electrical energy needs to become mechanical movement and sometimes, mechanical movement becomes electrical energy. But there is always a middleman taking a cut.

1. The motor is not 100% efficient. If your math says you need X Watts of mechanical power at the wheels to maintain speed, write the expression for the actual electrical power pulled from the battery.
2. When going down a steep hill, the car uses Regenerative Braking. Write the power equation for regen.
3. If you drive up a 100-meter hill and immediately drive down the exact same hill using regen braking to maintain a constant velocity, you will end up with less battery energy than you started with. List all the specific inefficiencies in the car's system that prevent you from recovering 100% of that potential energy.

Question 5

The Grand Ledger (Integration)

It's time to put all the blocks together.

1. Using all the individual components you defined, write the Grand Power Balance Equation for the car at any given instant t.
2. Based on your equation, define what it physically means for the car to be in Energy Positive, Energy Neutral and Energy Negative states. Give a brief real-world example of each happening on the road.
3. Look closely at your Grand Power Balance Equation. You would realise that there are terms out of our control. What is the single mathematical variable in this massive equation that the team has direct, immediate control over to manipulate the battery drain?

AGNIRATH

TAMING THE TELEMETRY

Question 1

Who doesn't like a black and white bear

Before you can optimize a race, you need to be able to understand and handle your data. Read up on Pandas dataframes and Numpy vectorisation.

1. What is the difference between .loc and .iloc? When would you use one over the other?
2. What does it mean to vectorize an operation in NumPy and why is it faster than a Python for-loop?
3. You are given a CSV file containing telemetry data from a test drive: columns include timestamp, velocity_kmh, battery_voltage, solar_irradiance_wm2 and motor_current_A. Some rows have missing or corrupt values (e.g., negative velocities, voltages above 160V, irradiance above 1200 W/m²). Perform the following operations and attach a screenshot of the code used to do so :
   a) Loads the CSV and performs an initial audit (shape, dtypes, null counts).
   b) Clean the data by removing or interpolating invalid rows, with clear justification for each decision.
   c) Compute power_input = solar_irradiance_wm2 × panel_efficiency × panel_area (assume reasonable constants and state them) and add this to the csv as a new column.

Question 2

Optimising optimally

Every constraint optimization problem has three sacred components:

1. Objective Function f(x) → What we brutally minimize (energy consumption)
2. Decision Variables x → What we control (velocity, battery allocation)
3. Constraints g(x) ≤ 0 → What we obey (safety, physics)

The physics of a solar car involve relationships between variables that aren't always linear and cannot be quantified in a simple manner. Read up on different types of optimisers you can use and understand how they differ from each other.

a) After reading what are the best optimisers you think we can use for our challenge. Explain your thought process.

Note : The optimiser should converge to a sane result in a decent amount of time(eg. for a medium sized dataset, it should converge in about 10-15 minutes). It should also converge properly while handling multiple equality and inequality constraints such as the following: Velocity at the start and end must be 0 (Equality constraint) and battery level should be above the threshold. They should also be easy to implement and modify.

b) Optimisers sometimes get stuck in local minimas instead of reaching a global minima. Let's say you are making your own optimiser, what measures would you take to avoid this?

c) Let's say the optimiser you have chosen has converged to an output which seems fine at first glance. What do you think is the best way to check if the result of an optimiser is indeed the optimal result? Explain your thought process.

Question 3

Computational Arms Race

Since we are **TEAM** Agnirath, it is important that the code we write efficient code and it can be read and understood by other members. (You should also be able to figure out what you wrote in your code if you open it 3 days later)

a) Below is a poorly written function that processes telemetry segments to find the one with the highest energy yield. It works correctly but violates almost every readability and performance principle.

```
def f(x, s):
    r = None
    mx = -1
    for i in range(len(x)):
        v = x[i]
        q = 0
        for j in range(len(s)):
            if s[j][0] == v:
                q = s[j][1] * s[j][2] * 0.21 * 4.6
        if q > mx:
            mx = q
            r = v
    return r
```

Explain all the changes you made and why you made them. Add a screenshot of the cleaned up code in the application.

b) Calculate the time complexity of the above code. Explain how you got to that result. Do the same for the below codes.

AGNIRATH

```python
def find_peak_irradiance_window(irradiance: list[float], k: int) -> float:
    n = len(irradiance)
    max_sum = 0

    for i in range(n - k + 1):
        window_sum = 0
        for j in range(i, i + k):
            window_sum += irradiance[j]
        if window_sum > max_sum:
            max_sum = window_sum

    return max_sum / k
```

```python
ef find_battery_conflicts(logs: list[tuple]) -> list:
    conflicts = []

    for i in range(len(logs)):
        for j in range(i + 1, len(logs)):
            t1, v1 = logs[i]
            t2, v2 = logs[j]
            if abs(v1 - v2) > 50 and abs(t1 - t2) < 10:
                conflicts.append((logs[i], logs[j]))

    return conflicts
```

Question 4

Time to shine

Now that you know the basics of file handling and efficient coding, lets test your skills.

a) Read up on coasting and explain why it is a good way to estimate Coefficient of drag and Rolling resistance.

b) You are given a csv file from our testing session (DM either of us for the data) where we were trying to stimate CDA and CRR. We do this by performing coasting.

- The file contains timestamps, velocity_ms, Gradient_deg, latitude, longitude.
- Load this data onto a python file
- Check the data for any faulty values, if there are any ignore them or interpolate them. Smoothen out the data before using it as the sensor values have some level of noise in them.
  (Hint : Look at rolling average).
- Perform curve fitting with the required equation and estimate values of CDA and CRR.
- Plot the fitted curve against the data to get an idea about its accuracy.
  (Hint : For curve fitting check out np.linalg.lstsq())

To learn more about Numpy and LinAlg watch this video :
https://www.youtube.com/watch?v=xvFZjo5PgG0

STAYING IN CONTROL

AGNIRATH

Think of the solar car in its environment as a dynamic system. It changes its state based on the environment and the control inputs you provide. You want to steer the car to finish the race efficiently.
In previous years, we relied on offline optimizers that generated a single velocity profile before the race began. This year, we are planning to build a Real-Time Online Optimizer. The car must continuously look at its current state, look at the road ahead and calculate a brand new optimal strategy on the fly. To do this, we need to enter the world of Control Theory.

Question 1

Defining the Dynamic System

Before we can control a system, we must translate it into mathematics. A system is defined by its State, its Controls and its Environment.

1. Define the specific State Variables (x) and Control Inputs (u) for our solar car.
2. We must frame our race as a constrained optimization problem. Define a clear Objective Function.
3. What are the possible Hard Constraints and Soft Constraints? Give one example of each for our car. Also, will your control be deterministic or stochastic? What does this depend on?

Question 2

PID & The CartPole Challenge

The most fundamental closed-loop controller in engineering is the PID Controller. It calculates an error and applies a correction.
The Coding Task: We want you to build intuition for how a controller actively fights instability. Using Python, write a simple PID controller script to balance a CartPole.
We have no issues in you using AI to write the animation code, but please do make your effort in writing the controller code.
(Hint: Search up on Wikipedia what this CartPole is)

1. Submit your Python script for the CartPole PID controller.
2. Play around with your Kp, Ki and Kd values. Based on your observations:
   - What physically happens to the cart if Kp is too high?
   - What happens if you completely remove the Kd term?
   - Why is a high Integral term dangerous over long periods of time?
3. Based on your tuning experience, why would a standard PID controller struggle to manage a solar car's speed efficiently when driving through weird routes?

AGNIRATH

Question 3

MPC (Must Plan Carefully)

PID controllers are reactive, but our car needs to be proactive. If there is a massive hill 3.7 kilometers away, the car needs to start saving energy now. This is where Model Predictive Control (MPC) comes in.

The Math: MPC uses a discrete mathematical model of the car's dynamics:

$x[k+1] = Ax[k] + Bu[k]$

At every time step, it solves an optimization problem to minimize a Cost Function (J) over a future horizon. A standard quadratic cost function looks like this: $J = \Sigma (x^T Q x + u^T R u$, where Q penalizes state errors and R penalizes control effort.

1. If $x[k+1] = Ax[k] + Bu[k]$ represents the car, what do the matrices A and B represent physically?
2. Look at the Cost Function J. Tell me how you would redefine the loss equation to penalize rapid changes in throttle.
3. Mathematically, why does MPC inherently handle physical limits better than a standard PID controller?

Question 4

Multi-Variable Madness & Escaping the Trap

1. In a MIMO(Multiple Input Multiple Output) system, your MPC must balance multiple competing objectives simultaneously with tracking a target velocity and minimizing battery drain being the main ones.

    - Detail the potential mathematical and physical issues that arise when tuning the cost function weights (Q and R matrices) for these competing variables.
        - How do you rectify these issues?

2. You have set two strict Hard Constraints in your MPC optimizer:
    - You must maintain a minimum speed of 60 km/h to avoid impeding highway traffic.
    - You must never exceed a battery current draw of 50 Amps to prevent the Battery Management System from tripping and shutting off the car.

    Suddenly, an unforecasted, massive headwind hits the car while climbing a steep hill. To maintain 60 km/h, the physics model dictates the car must draw 65 Amps.

    a. What will a standard mathematical MPC solver do at this exact moment?
    b. Mathematically, how do you reformulate your MPC algorithm to escape this trap so the model doesn't just crash?

THE STRATEGIST'S CRUCIBLE

AGNIRATH

Forget theoretical ideals; on the track, survival is the only metric that matters. We don't care about perfect scenarios. Your job as an Agnirath Strategy Member would be to navigate these issues. Essentially, we're putting you in our seats to see if you can protect your battery budget while pushing for the finish line.

Note: These are difficult problems. Even if you cannot fully solve it, do it as much as you can. Submit your workings and write a brief report on your intuition.

Question 1

Terrain and Constrained Optimization

The Setup: You are analyzing a 5 km valley segment:

- Segment 1: 2.5 km steep downhill (slope -α).
- Segment 2: 2.5 km steep uphill (slope +β).

To meet your stage deadline, you must complete this 5 km segment with a target average speed of v_0 = 75 km/h. You are bound by strict race regulations: v_{min} = 60 km/h and v_{max} = 100 km/h.

The mechanical power required to move the car is: $P_{mech}(v, \theta) = k * v^3 + P_base + m * g * v * \sin(\theta)$

Your electrical systems are imperfect:

- When $P_{mech} > 0$ (Motoring), the battery drain is $P_{batt} = P_{mech} / \eta_m$ (Assume η_m = 0.90).
- When $P_{mech} < 0$ (Braking), energy flows back into the battery at $P_{batt} = \eta_r * P_{mech}$ (Assume η_r = 0.70).

Strategy A (Strict Cruise Control): The driver locks the cruise control at v_0 = 75 km/h for the entire 5 km. The downhill slope -α is steep enough that P_{mech} is negative, forcing the car to use regen braking to avoid speeding up.

Strategy B (Variable Velocity): The driver goes down the hill at a faster constant speed v_1 (> 75 km/h) and climbs the hill at a slower constant speed v_2 (< 75 km/h). They choose v_1 and v_2 such that they perfectly meet the overall time target.

The Task:

1. The Math: Formulate the total battery energy equations E_A and E_B. Using these equations, mathematically prove which strategy consumes less net battery energy for small speed deviations from v_0 = 75 km/h.
2. The Boundary

   "Bro, this is so obvious. Why would Strategy B not always be better? Just let gravity do the work and don't touch the brakes." — Saksham when Hafiz proposed this problem to him.

Saskham's intuition makes sense on the surface, but he is not fully right.
Prove Saksham wrong: What is the exact mathematical boundary where Strategy B stops saving energy and actually becomes worse than Cruise Control?
3. The Intuition: When you complete your mathematical proof, you will notice that Strategy B's superiority relies entirely on some mathematical formulation.

- What real-world engineering concept does this represent? How does this explain Saksham's intuition?
- Imagine a world in which such said formulation is not an issue. What would be the implications of this fictional scenario in that world?

Question 2

The Blind Loop Dilemma

The Setup: It is 3:30 PM. The race ends strictly at 5:00 PM (You have T_{max} hours remaining). You have just arrived at a checkpoint. An optional loop is exactly L km. Your battery currently sits at $E_{current}$, but regulations and battery health dictate that you cannot drop below E_{min}.
Your car's mechanical power equation is $P_{mech} = k * v^3 + P_{losses}$. The electrical power drawn from the battery is $P_{draw} = P_{mech} / \eta$. Based on late-afternoon irradiance data, your array provides a constant solar intake of P_{solar}.

The Task:

1. The Algebra: Let N be the strictly integer number of loops you choose to complete. Before plugging in any values, algebraically formulate the two primary constraints, which are time constraint and energy constraint that dictate whether N loops are feasible at a constant velocity v. Isolate v in both inequalities to explicitly show how time forces a minimum required speed, while energy forces a maximum allowable speed.
2. The Numerical Solution: We highly recommend using Python to simulate this environment and use numerical methods to find the optimal solution. Using the following values, find the maximum integer number of loops N you can complete and the exact target velocity v required to do so:
   - T_{max} = 1.5 hours, L = 30 km, $E_{current}$ = 1.5 kWh, E_{min} = 0.2 kWh and P_{solar} = 250 W

   (Assume some constant values for k, η and P_{losses} and explain your reasoning).
3. The Final Call: Assuming your numerical solution yields a valid range of feasible speeds, where $v_{min} \leq v \leq v_{max}$, to complete the maximum number of loops, you must now make a decision. What exact target velocity within this valid range do you instruct the driver to maintain? Explain the reasoning behind your choice and describe why picking other valid speeds in that range would put the car at a massive disadvantage for the next day.

THE FINAL CHALLENGE

AGNIRATH

This is the ultimate test of your application. We are not here to study theory, but we are looking for engineers who can translate strategy into functional code. If you are short on time, submit a highly detailed architectural design and pseudocode, but a working repository is what we expect from you here.

The Race Context: The Sasol Solar Challenge

You are simulating Day 2 of a simplified version of the Sasol Solar Challenge.

- The Route: We are driving from Sasolburg to the control stop in Zeerust, which is also where the day ends.
- The Clock: The race starts at exactly 8:00 AM. You must cross the final timing line for the day at exactly 5:00 PM or earlier.
- The Rules of Distance Maximization: In Sasol, the goal is not just to reach the finish line, but the goal is to maximise your distance travelled.
- Base Arrival: When you reach Zeerust, you must perform a mandatory 30-minute control stop and the car is stationary here.
- The Loops: After the 30 minutes, you enter the Distance Maximization phase. You can drive an optional 35 km loop around Zeerust. Assume this 35 km loop has straight-line physics (do not worry about cornering dynamics, elevation changes, or traffic for the loops).
- The Loop Return: Every time you complete a loop and return to base, you must perform an additional 5-minute mandatory stop before you can start another loop.
- The End of Day: All driving must cease by 5:00 PM. If you are caught out on a loop at 5:00 PM, that loop's distance does not count.
- The Survival Constraint: You cannot exhaust the battery. To survive Day 3, your car must end the day at 5:00 PM with a Battery State of Charge (SOC) of strictly ≥ 20%.

AGNIRATH

P**hase 1: The Cartographer (Data Pipeline)**

Before you can optimize the car, you need to map the environment.

1. **The Route API:**

- Identify a suitable public API (e.g., Open Elevation, Google Maps, OSRM, SRTM) to map the route from Sasolburg to Zeerust.
- Retrieve and save the GPS coordinates, Altitude and Direction along the route.
- Compute the Slope of the road between each point.
- Question: What spatial resolution did you choose and why is this resolution appropriate for a race simulation? Save this data as a CSV.

2. **The Solar Model**: Instead of scraping live weather data, you will build a mathematical solar model. Write a function that calculates the incident solar irradiance based on the time of day using a Gaussian curve.

- Assume the peak solar irradiance is roughly 1073 W/m², occurring at exactly Solar Noon (12:00 PM).
- Assume the standard deviation of the daylight curve is 11,600 seconds.
- Do not forget to factor in Panel Efficiency and Area! (We hope you remember the values from the orientation session).

Phase 2: The Strategist (Optimization Engine)

This is the core task. Build a predictive strategy model that dictates the exact velocity profile for the car throughout the entire day.

The Objective: Maximize the total valid distance covered (Base Route + N full loops)

Strategic Hint (The Two-Model Approach): Because the base route has dynamic elevation and the loops have straight-line physics, you will likely need to build two distinct optimization models: one to get you to Zeerust and a second to maximize the repetitive loops..

The Constraints:

- Must reach Zeerust, serve the 30-minute stop and complete all loops before 5:00 PM.
- Mandatory 5-minute stops between loops.
- Battery SOC must be ≥ 20% at all times.
- Velocity must remain within legal road bounds.
- Acceleration must be bounded to realistic physical limits for a solar car.

Implementation Details:

- You may use Python or C++.
- You can use built-in optimization libraries (e.g., SciPy minimize, CasADi) or write your own numeric solver.
- Keep the code modular. Separate your data-fetching, your physics calculations and your optimizer into different files or functions.
- Performance Cap: Your model should converge and output a solution in under 10 to 15 minutes for a medium-sized dataset.

Phase 3: The Analyst (Outputs & Visualization)
Your code must output the results visually.
1. Plot Generation: Generate the following plots (graphed against Time or Distance):

- Velocity Profile: Show how fast the car is moving. (We should clearly see the periods where velocity is 0 km/h for the control stops).
- State of Charge (SOC) Profile: Show the battery profile throughout the day.
- Acceleration Profile: Show the rate of change of velocity.
- Other Useful Plots: Add any extra plots you feel prove your strategy works

2. Insights & Analysis: Explain what the code discovered. Walk us through your strategic choices and why the optimizer behaved the way it did.

What to Submit (The Checklist):

Please submit a link to a Public GitHub Repository containing:

- Your modular, readable and well-commented source code (Python or C++).
- The generated CSV of your route data.
- The exported Plots (PNGs/JPGs) of your Velocity, SOC, Acceleration and any other profiles. Have all plots that you want to show us ready since analysis of graphs is important.
- A comprehensive README.md detailing your system architecture, your mathematical assumption and your analytical insights. Mention all these in your application as well.